CDN stands for Content Delivery Network, they are primarily designed to cache assets all over the world so that when a browser requests the asset, a cached copy will be geographically close to that browser. If you are serving assets directly from your Rails server in production, the best practice is to use a CDN in front of your application.

A common pattern for using a CDN is to set your production application as the "origin" server. This means when a browser requests an asset from the CDN and there is a cache miss, it will grab the file from your server on the fly and then cache it. For example if you are running a Rails application on `example.com` and have a CDN configured at `mycdnsubdomain.fictional-cdn.com`, then when a request is made to `mycdnsubdomain.fictional- cdn.com/assets/smile.png`, the CDN will query your server once at `example.com/assets/smile.png` and cache the request. The next request to the CDN that comes in to the same URL will hit the cached copy. When the CDN can serve an asset directly the request never touches your Rails server. Since the assets from a CDN are geographically closer to the browser, the request is faster, and since your server doesn't need to spend time serving assets, it can focus on serving application code as fast as possible.

4.4.1 Set up a CDN to Serve Static Assets

To set up your CDN you have to have your application running in production on the internet at a publically available URL, for example `example.com`. Next you'll need to sign up for a CDN service from a cloud hosting provider. When you do this you need to configure the "origin" of the CDN to point back at your website `example.com`, check your provider for documentation on configuring the origin server.

The CDN you provisioned should give you a custom subdomain for your application such as `mycdnsubdomain.fictional-cdn.com` (note fictional-cdn.com is not a valid CDN provider at the time of this writing). Now that you have configured your CDN server, you need to tell browsers to use your CDN to grab assets instead of your Rails server directly. You can do this by configuring Rails to set your CDN as the asset host instead of using a relative path. To set your asset host in Rails, you need to set `config.action_controller.asset_host` in `config/production.rb`:

```
config.action_controller.asset_host = 'mycdnsubdomain.fictional-
cdn.com'
```

You only need to provide the "host", this is the subdomain and root domain, you do not need to specify a protocol or "scheme" such as `http://` or `https://`. When a web page is requested, the protocol in the link to your asset that is generated will match how the webpage is accessed by default.

You can also set this value through an environment variable to make running a staging copy of your site easier:

```
config.action_controller.asset_host = ENV['CDN_HOST']
```

Note: You would need to set `CDN_HOST` on your server to `mycdnsubdomain .fictional-cdn.com` for this to work.

Once you have configured your server and your CDN when you serve a webpage that has an asset:

```
<%= asset_path('smile.png') %>
```

Instead of returning a path such as `/assets/smile.png` (digests are left out for readability). The URL generated will have the full path to your CDN.

```
http://mycdnsubdomain.fictional-cdn.com/assets/smile.png
```

If the CDN has a copy of `smile.png` it will serve it to the browser and your server doesn't even know it was requested. If the CDN does not have a copy it will try to find it a the "origin" `example.com/assets/smile.png` and then store it for future use.

If you want to serve only some assets from your CDN, you can use custom `:host` option your asset helper, which overwrites value set in `config.action_controller.asset_host`.

```
<%= asset_path 'image.png', host: 'mycdnsubdomain.fictional-cdn.com' %>
```